

CHAPTER III – METHODOLOGY

A. PROJECT DESCRIPTION

What is it?

This section gives a **concise but informative overview** of your system. It tells the reader:

- What the system is for
- Who uses it
- What it does (functions/modules)
- Where it's deployed
- What tools or languages were used to develop it

Think of it as the **summary profile** of your actual system.

Steps to Write:

1. State what the system is and its purpose.
2. Identify the target users (admin, staff, students, etc.).
3. List the core functions (borrow, return, stock-in/out, reports).
4. Mention the development tools used (PHP, MySQL, etc.).
5. Specify the environment (LAN, web, mobile, offline).

✅ Full Sample:

The **Tools Inventory System** was developed to automate and streamline inventory management processes within the **BIT Automotive Program** at Iloilo Science and Technology University Dumangas Campus. The system is intended to be used by **authorized users**, such as laboratory staff and administrators, to manage the issuance and return of tools, monitor stock-in and stock-out transactions, and generate comprehensive inventory reports.

Built using **PHP** for the front-end, **MySQL** for the database, and hosted locally using **XAMPP**, the system runs on a **local area network (LAN)**. This ensures that the application is accessible within the confines of the laboratory, ensuring data privacy, operational efficiency, and real-time updates of tool availability and usage.

Ⓢ. PROJECT DEVELOPMENT PROCESS

📖 What is it?

This section explains the **step-by-step approach** used to develop your system—from planning to deployment. It shows that your project followed a **systematic software development method**.

🧠 Method Used: Prototyping Model

- The Prototyping Model is an iterative approach.
- A **prototype** (early version of the system) is built and shown to users.
- Users give feedback → the system is **revised and improved** continuously until final.

Steps to Write:

1. State what methodology was used (e.g., Prototyping Model).
2. List each phase:
 - Requirements Gathering
 - Quick Design
 - Prototype Development
 - User Evaluation
 - Refinement
 - Testing
 - Deployment
3. Describe the **activities** done in each phase.

Full Sample:

The development of the Tools Inventory System followed the **Prototyping Model**, an iterative software development approach that emphasizes early user involvement and continuous refinement.

- **Requirements Gathering:** The developers conducted interviews with laboratory staff, faculty members, and the program coordinator to understand the current inventory process and identify issues in the manual system.
- **Quick Design:** Based on the gathered data, a quick design was created showing basic layouts and modules for tool registry, stock-in/out, borrowing, returning, and report generation.
- **Prototype Development:** A working prototype was built using PHP and MySQL. Initial modules included manual input of tools, simple stock tracking, and user login.
- **User Evaluation:** The prototype was demonstrated to the staff. Feedback was collected on usability, missing features, and interface flow.
- **Refinement:** The interface was improved, new validations were added, and transaction logs were made more robust based on user feedback.

- **Testing:** The system was tested using sample data to ensure all modules worked as intended, including corner cases such as duplicate borrowing or stock shortages.
 - **Deployment:** The final version was deployed in the automotive laboratory. Users were trained, and the system was evaluated using ISO 25010 criteria.
-

🕒 UML DIAGRAMS

📖 What is it?

UML (Unified Modeling Language) diagrams visually **represent the structure and behavior** of your system. They help others **understand how the system works** without reading code.

📋 Diagrams to Include:

1. Use Case Diagram:

- Shows the users (actors) and what they can do (use cases).
- E.g., Admin → Manage Users, Generate Reports

2. Activity Diagram:

- Shows the flow of processes (like a flowchart).
- E.g., Log in → Select Tool → Borrow → Confirm → Generate Receipt

3. Class Diagram:

- Shows database structure (tables, attributes, relationships).
- E.g., tblUser, tblTools, tblBorrow linked with foreign keys

4. Sequence Diagram:

- Shows how components interact over time.
- E.g., User → Login Form → PHP → Database

5. System Architecture Diagram:

- Shows physical deployment: PCs, LAN, database server, etc.

✓ Tools You Can Use:

- draw.io (free)
- Lucidchart
- Creately
- Microsoft Visio (if available)

📌 Sample Use Case (Textual Description):

The **Use Case Diagram** shows two actors: Administrator and Custodian. Both can manage the tool registry, record borrowing and return transactions, and generate reports. However, only the Administrator has access to the user account management module.

⑩. REQUIREMENT ANALYSIS

📖 What is it?

This section identifies the **minimum hardware and software** required to run your system smoothly.

It ensures your system can be **reproduced or deployed** successfully elsewhere.

📝 Steps to Write:

1. List **hardware specs**: CPU, RAM, storage, network.
2. List **software requirements**: OS, server tools, browsers, editors.
3. Mention **any dependencies**: XAMPP, PHP version, MySQL, etc.

✓ **Full Sample:**

Hardware Requirements

- **CPU:** Dual Core 1.8 GHz or higher
- **RAM:** 8 GB minimum
- **Storage:** 120 GB SSD or higher
- **Input Devices:** Mouse, Keyboard
- **Display:** Monitor with at least 1280x720 resolution
- **Network:** LAN (Local Area Network)

Software Requirements

- **Operating System:** Windows 10 or higher
- **Development Tools:** Sublime Text
- **Web Server Environment:** XAMPP (Apache, PHP 8.2+, MySQL)
- **Database Management:** MySQL
- **Browser for GUI:** Google Chrome or Mozilla Firefox (latest version)

🔊. SYSTEM EVALUATION (ISO 25010)

What is it?

This section describes how the **quality of your system** will be assessed using the **ISO 25010 software quality standard**.

This standard has **8 evaluation criteria**.

 Evaluation Criteria and Meaning:

Criteria	Description
Functionality	Does the system meet the requirements and perform its tasks?
Usability	Is the system easy to use and understand?
Reliability	Does it work consistently without errors or crashes?
Performance	Does it respond quickly and run efficiently?
Maintainability	Can the code be fixed, updated, or expanded easily?
Portability	Can it be installed or moved to other machines easily?
Security	Does it protect data and restrict access?
Compatibility	Does it work with the OS, browser, or network where it is installed?

 Rating Scale:

Rating	Description
4.21–5.00	Excellent
3.41–4.20	Very Good
2.61–3.40	Good
1.81–2.60	Poor
1.00–1.80	Very Poor

✅ Sample Evaluation Use:

The system was evaluated by 5 users (faculty and IT experts) using an ISO 25010-based survey. Each criterion was rated from 1 to 5 using a Likert scale. The average score was computed per category and interpreted using the standard scale.

📊. DATA GATHERING & STATISTICAL TREATMENT

📖 What is it?

This part explains **how feedback was collected** and **how the results were analyzed**. It applies especially to your **system evaluation survey**.

📋 Steps to Write:

1. Identify your **respondents** (e.g., 5 IT Experts, 5 Domain Experts).
 2. Describe your **instrument** (ISO 25010-based questionnaire with 8 categories).
 3. Use a **Likert Scale** (1–5) for ratings.
 4. Analyze using **descriptive statistics** (mean).
 5. Interpret using the scale (Excellent, Very Good, etc.).
 6. Present results in a **table**.
-

✅ Full Sample:

Instrument:

The evaluation used a questionnaire based on **ISO 25010**, consisting of 24 items grouped under 8 quality characteristics. Responses were recorded using a **5-point Likert Scale**.

Respondents:

A total of **10 evaluators** (5 IT experts and 5 domain experts) were purposively selected to assess the system's functionality, usability, and maintainability.

Statistical Treatment:

The mean was computed for each category using this formula:

$$\text{Mean} = \frac{\text{Sum of Ratings}}{\text{Number of Respondents}}$$

Mean=Number of RespondentsSum of Ratings

Interpretation Scale:

Mean	Interpretation
------	----------------

4.21–5.00	Excellent
-----------	-----------

3.41–4.20	Very Good
-----------	-----------

Sample Table:

Category	Mean	Interpretation
Functionality	4.80	Excellent
Usability	4.60	Excellent
Maintainability	4.40	Excellent
Security	4.10	Very Good
Compatibility	4.00	Very Good
